NeuFlow v3: Queryable Optical Flow with Calibrated Uncertainty for Region-Directed Perception

Shriman Raghav Srinivasan
Department of Electrical and Computer Engineering
Northeastern University
Boston, MA, USA

***Abstract*—Optical flow networks emit a dense displacement field at a single fixed resolution. Many consumers of flow, including image registration, sparse tracking and survey mosaicking, require a few hundred correspondences at locations they select rather than a value at every pixel. We present NeuFlow v3, which retains the NeuFlow v2 pipeline through its coarse flow stage and replaces only the final convex upsampler with an implicit decoder evaluated at arbitrary continuous coordinates. Cost then scales with the number of queries, $O(N)$, rather than with image area, $O(H{\times}W)$. On held-out driving sequences the decoder is 2.6% less accurate than the upsampler it replaces, from a model with 13% fewer parameters. In exchange it provides three capabilities the fixed-resolution design cannot express: repeat queries against a cached frame at $27\times$ lower cost, per-query calibrated confidence, and evaluation at coordinates between input pixels. The confidence signal is the more consequential of these: permitting the model to abstain on its least confident 20% of queries yields 1.06 px mean error against the baseline's 2.32 px over the remaining 80% of the frame. We further characterise region-directed inference for fast-moving platforms, showing that processing 14% of the frame costs 0.034 px and that the context margin a cropped region requires is set by inter-frame displacement rather than image size, a relation we verify at two motion scales differing by a factor of three.**

***Index Terms*—optical flow, implicit neural representation, uncertainty estimation, edge computing, region of interest**

I. Introduction

Optical flow estimation has converged on a common output contract: given two frames, produce a displacement vector for every pixel, at the resolution of the input. Modern real-time networks such as NeuFlow v2 [1] attain this efficiently enough for embedded deployment. The contract itself, however, is rarely questioned, and it does not match what several important consumers of flow actually require.

Image registration selects a few hundred well-textured points and needs correspondences at those points. Sparse feature tracking needs flow at detected keypoints. Survey mosaicking needs matches within overlap regions, frequently at sub-pixel positions. In each case a dense field is computed, and the great majority of it is discarded. On constrained hardware, computing 479,232 correspondences in order to consume 800 of them is the difference between operating in real time and not.

The fixed-resolution contract also forecloses two operations outright. A consumer cannot request flow at a position between input pixels, and it cannot request additional correspondences on a frame the network has already processed without paying for the entire computation again.

This paper asks whether the final stage of a real-time flow network can be made queryable without surrendering accuracy or throughput, and reports what such a change costs and what it buys. Our contributions are:

1) An implicit decoder that replaces NeuFlow v2's convex upsampler, answering flow queries at arbitrary continuous coordinates in $O(N)$, with sparse output provably identical to dense output at matching coordinates (0.00057 px maximum deviation).
2) A per-query uncertainty estimate, calibrated across a $15\times$ range, which enables selective prediction: $2.2\times$ lower error than the baseline over 80% of the frame.
3) A characterisation of region-directed inference for fast platforms, including a design rule relating the required context margin to platform speed and frame rate, verified across two domains whose motion scales differ by a factor of three.
4) An explicit accounting of what the change costs: 2.6% mean accuracy on the training domain, with the mechanism identified.

II. Related Work

Real-time optical flow. NeuFlow v2 [1] attains real-time performance on embedded hardware through a shallow CNN backbone, global matching at $1/16$ resolution, and lightweight recurrent refinement, followed by a learned convex upsampler. We adopt its front end unchanged and modify only the output stage.

Implicit neural representations for dense prediction. AnyFlow [2] introduced arbitrary-scale optical flow by predicting convex combination weights over a neighbourhood of coarse flow values, conditioned on a query coordinate. It is built on RAFT [3] and reports no inference timings, leaving open whether the approach is viable under real-time constraints. Our decoder adopts the convex-weight formulation and asks precisely that question. InfiniDepth [4] contributes the gated hierarchical fusion of multi-scale features that we use to condition the head.

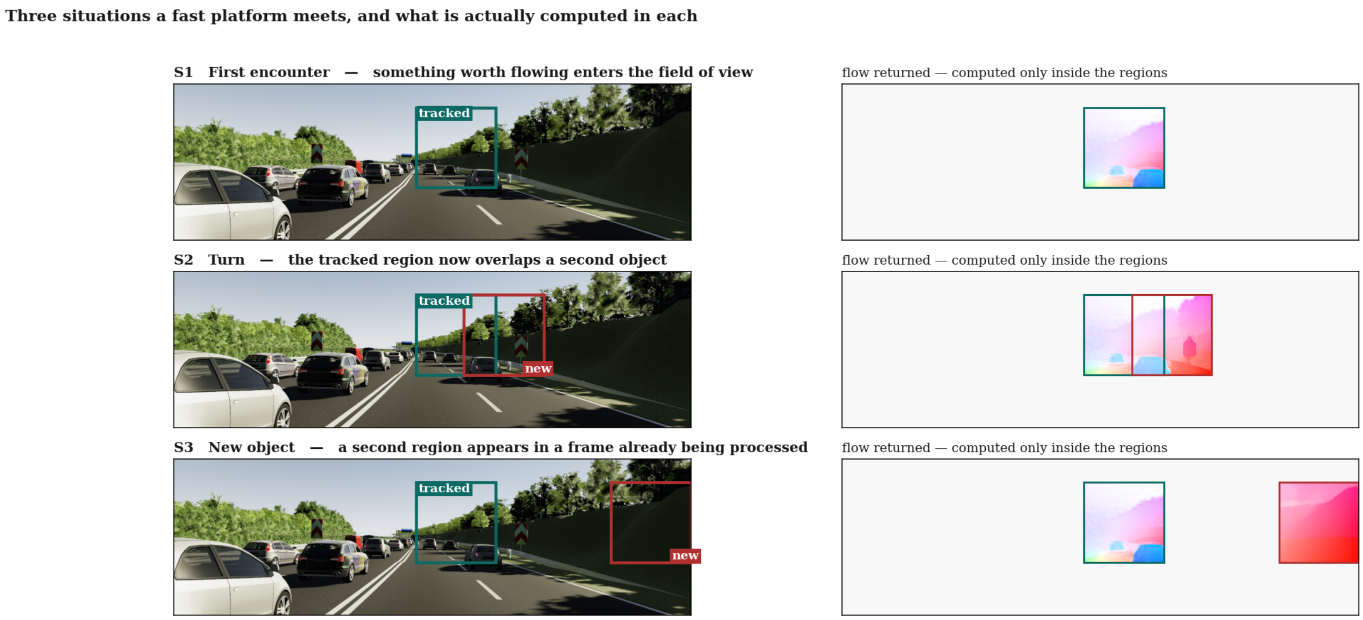

Fig. 1. Three situations a fast platform encounters, on held-out driving frames. Left: the frame with the region or regions a tracker would nominate. Right: the flow the decoder actually returns, computed only inside those regions. Top, a first encounter; middle, a turn in which the tracked region comes to overlap a second object; bottom, a new object appearing in a frame already being processed, which the decoder answers from cached state.

Uncertainty in dense prediction. Predicting a per-pixel error scale under a Laplace likelihood is standard in depth and flow estimation [5]. We apply it per query rather than per pixel, which makes the estimate available for correspondences the consumer selects, and evaluate it by selective prediction rather than by correlation alone.

III. Method

A. Inherited Pipeline

NeuFlow v2 extracts features at $1/8$ and $1/16$ resolution with a shallow CNN, applies cross-attention and global matching at $1/16$ to establish an initial flow, refines it recurrently (one iteration at $1/16$, eight at $1/8$), and upsamples the resulting $1/8$-resolution flow with a learned convex combination over a 3×3 neighbourhood.

We retain everything through the coarse flow, with all learned weights and all normalisation statistics frozen at their published values. We verify this per checkpoint: all 137 tensors shared with the baseline are bit-identical, so the comparisons that follow isolate the decoder.

B. Implicit Decoder

The decoder operates in two phases, which is what makes repeat queries cheap.

Phase 1, once per frame pair. The frozen pipeline produces the coarse flow and feature maps, which are cached as state.

Phase 2, once per query batch. For a query coordinate (x, y) the decoder bilinearly samples 3×3 windows from four sources: $1/8$ context, $1/8$ matching features, $1/16$ features, and frame-1 features warped by the coarse flow. These are fused by a gated hierarchical MLP following InfiniDepth, and the result conditions a head predicting weights over ten candidates: the nine coarse-flow values in the local neighbourhood, plus a bilinear sample at the query position.

The output is a convex combination,

$$\mathbf{f}(x,y) = \sum_{i=1}^{10} w_i(x,y)\,\mathbf{c}_i(x,y), \quad \sum_i w_i = 1,\ w_i \geq 0, \tag{1}$$

where $\mathbf{c}_i$ are the candidate displacements and w_i are softmax weights. Boundedness matters: the prediction cannot leave the convex hull of locally supported motion, so the decoder cannot hallucinate displacement its inputs do not support.

We initialise the head so that the softmax concentrates on the bilinear candidate. An untrained decoder is therefore equivalent to bilinear upsampling (0.011 px maximum deviation), and training can only depart from that known-good operating point by learning.

C. Uncertainty Head

An optional eleventh output predicts a per-query error scale b, trained under a Laplace likelihood jointly with the flow loss,

$$\mathcal{L}_u = \frac{|\mathbf{f} - \mathbf{f}^*|}{b} + \log b. \tag{2}$$

It costs no measurable inference time and attaches a confidence value to every returned correspondence.

D. Interface

```
st = model.infer_coarse_state(img0, img1)

# arbitrary coordinates
flow = model.decode_queries(st, query_coords=q)

# or any output resolution
```

TABLE I
ACCURACY ON HELD-OUT DRIVING SEQUENCES

Model	Training data	EPE	1px	3px
NeuFlow v2	FlyingThings	**2.324**	**77.6**	**89.8**
NeuFlow v3	FlyingChairs	2.500	72.8	87.9
NeuFlow v3	+ VKITTI2	2.398	75.7	88.9
NeuFlow v3	+ Sintel	2.392	75.8	89.0
NeuFlow v3	+ uncertainty	2.384	76.1	89.0

TABLE II
LATENCY PER FRAME PAIR

Mode	Latency	Note
v2, full frame	33.3 ms	its only mode
v3, first sparse query	34.1 ms	level with v2
v3, repeat query	**1.25 ms**	27× **cheaper**
v3, dense output	37.1 ms	not the intended mode

```
flow = model.decode_queries(st, tgt_h, tgt_w)

# with per-query confidence
flow, b = model.decode_queries(st, q,
                return_uncertainty=True)
```

Coordinates are continuous: $(312.7, 188.2)$ is as valid an argument as $(312, 188)$.

IV. Experimental Setup

Evaluation. Virtual KITTI 2 [6] Scenes 18 and 20: 1,174 frame pairs, 460,573,660 valid pixels, per-pixel metrics. Both scenes are excluded from every training set, enforced in the data loader and verified at pair and frame level before each run. Cross-domain evaluation uses TartanAir [7] aerial sequences, which no model in this study was trained on.

Training. All runs are generated from a single template so that any two differ in exactly one variable: seed 1234, batch size 16, learning rate 2×10^{-4} under a OneCycle schedule, loss weighting $\gamma = 0.8$, 4,096 supervision queries per image, refinement schedule matching the evaluation configuration, 100,000 steps, backbone frozen throughout.

Hardware. NVIDIA RTX 4060 laptop GPU, fp16, 384×1248 input unless stated.

V. Results

A. Accuracy

Table I reports the central cost. Every v3 configuration is less accurate than the baseline. The best is 2.6% worse on mean end-point error and 1.5 points worse on 1-pixel accuracy; the like-for-like comparison, trained on FlyingChairs alone as the baseline was trained on FlyingThings alone, is 7.6% behind. The model is 13% smaller (7.83 M against 9.03 M parameters).

The ordering across training mixes is monotonic, though the last three configurations differ by less than 0.02 px and we do not draw conclusions from their ranking under a single seed.

TABLE III
ERROR OF THE ACCEPTED SET AT EACH COVERAGE LEVEL

Coverage	NeuFlow v3	NeuFlow v2
20%	**0.480**	2.324
40%	0.688	2.324
60%	0.798	2.324
80%	**1.058**	2.324
100%	2.266	2.324

B. Compute

The sparse path comprises a 32.8 ms coarse pass, inherited unchanged, and a 1.25 ms decode. Because global matching requires whole-image context, the coarse pass cannot be avoided; it accounts for 96% of a first query, which is why a first query costs what a baseline frame costs.

The decisive property is state reuse. The baseline retains nothing between calls, so a second question about the same frame requires recomputing everything. Decode latency is flat in the query count over the operating range (1.254 ms at N=800, 1.285 ms at N=2,048), being bounded by kernel launch overhead rather than arithmetic, so 2,048 points cost what 800 do.

C. Selective Prediction

Mean error over every pixel is a single operating point, and it is the only one the baseline has, because it cannot indicate which of its outputs to trust. Given a per-query confidence, v3 has a curve (Table III, Fig. 2). Discarding the least confident 20% of queries reduces error on the remainder to 1.058 px against the baseline's 2.324 px, more than twice as accurate over 80% of the frame. Retaining only the most confident fifth gives 0.480 px.

This is not a statistical artefact of selection: for registration and mapping the consumer chooses a few hundred points regardless, so declining to answer where the model is uncertain costs nothing it would otherwise have used.

D. Calibration

Over 2,348,000 queries, actual error rises monotonically with predicted scale across all five bins, from 0.480 px in the most confident to 7.100 px in the least, a factor of 15, with Pearson correlation 0.318. The signal is directly usable for weighting correspondences, rejecting unreliable matches, or allocating further queries. The baseline emits flow alone and offers no comparable quantity.

E. Region-Directed Inference

A platform that cannot afford full-frame flow must process only the region that matters. Table IV quantifies this for a 192×192 region of interest. Retaining full resolution and processing roughly 14% of the frame area costs 0.034 px and runs 4.4× faster.

The context margin is not optional. Without one, error rises by 0.432 px and 24% of large-motion pixels fail outright, because global matching loses the context it uses to locate

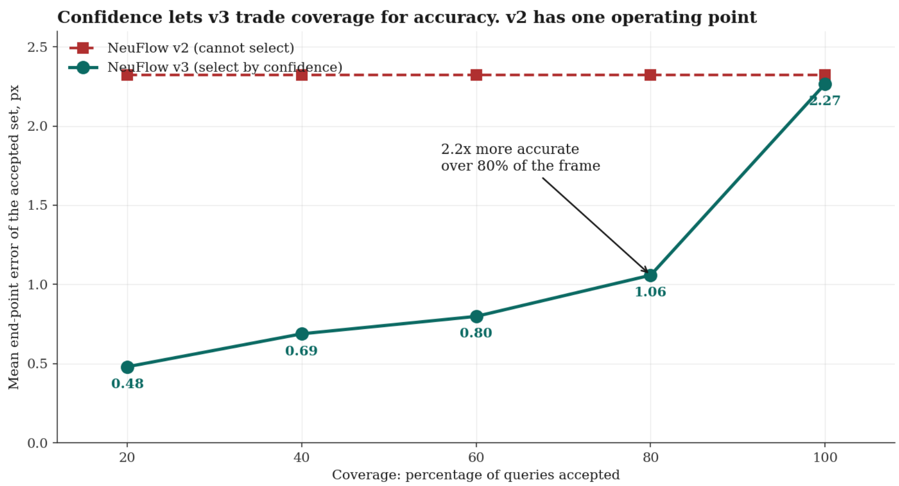

Fig. 2. Risk-coverage behaviour. The baseline has one operating point because it cannot indicate which outputs to trust; per-query confidence gives the decoder a curve.

TABLE IV
PROCESSING A REGION INSTEAD OF THE FRAME

Region	Latency	Speedup	EPE
Full frame	33.3 ms	1.0×	0.657
ROI, no margin	7.8 ms	4.3×	1.089
ROI +32 px margin	**7.6 ms**	4.4×	**0.691**
ROI +64 px margin	8.1 ms	4.1×	0.667
ROI +128 px margin	9.9 ms	3.4×	0.655

TABLE V
REQUIRED MARGIN AT TWO MOTION SCALES, MEASURED WITH THE BASELINE NETWORK. MOTION, MARGIN AND COST IN PIXELS.

Domain	Motion	Margin	Cost	At m=0
Driving (VKITTI2)	26.6	32	+0.034	+0.432
Aerial (TartanAir)	9.26	8	+0.120	+0.420

distant correspondences. We emphasise that this technique applies to any flow network including the baseline; it is a property of the architecture family, not of the queryable decoder.

F. The Margin Rule

If the margin requirement is set by displacement rather than by image size, it should scale with motion. We test this on aerial sequences whose mean displacement is a third of the driving case. Both domains begin from essentially the same penalty with no margin, 0.43 and 0.42 px, and both shed most of it once the margin reaches approximately one frame of motion (Table V, Fig. 3), giving

$$\text{margin} \approx \text{inter-frame displacement} \approx \frac{\text{platform speed}}{\text{frame rate}}. \tag{3}$$

This can be evaluated before any flow is computed. The agreement is close up to that point and the residuals beyond it differ by scene, so the relation predicts the scale of the required margin rather than the exact curve.

TABLE VI
COST OF A NEW REGION APPEARING MID-FRAME

Policy	Cost	EPE
v3, sparse queries	**1.68 ms**	2.10
v3, dense region	4.36 ms	1.77
v2, new cropped pass	7.29 ms	3.60
v3, new cropped pass	8.23 ms	3.61

G. Answering an Unanticipated Query

A platform frequently discovers a second region of interest after a frame is already in flight. Because the coarse state is cached, the decoder answers for 1.68 ms. A cropped pipeline must execute an entire new pass, and is markedly less accurate doing so because a fresh crop has again lost global context (Table VI). Two disjoint crops cost more than one full frame, giving a simple policy: crop once, or not at all.

H. Cross-Domain Comparison

On aerial sequences (Fig. 4) the decoder is ahead of the baseline at all seven margin settings tested, by 0.004 to 0.029 px (0.777 against 0.781 at full frame). We note this is the only unconfounded direct comparison in the study: every driving result is measured on a domain the decoder trained on and the baseline did not, whereas neither model has seen aerial imagery. The effect is small and measured on 40 pairs under one seed; it is consistent across seven settings, but the decoder remains slightly worse on large-motion failures (7.2% against 6.5%).

VI. DISCUSSION

The contribution is an operating point rather than a leaderboard position. The decoder trades 2.6% of mean accuracy

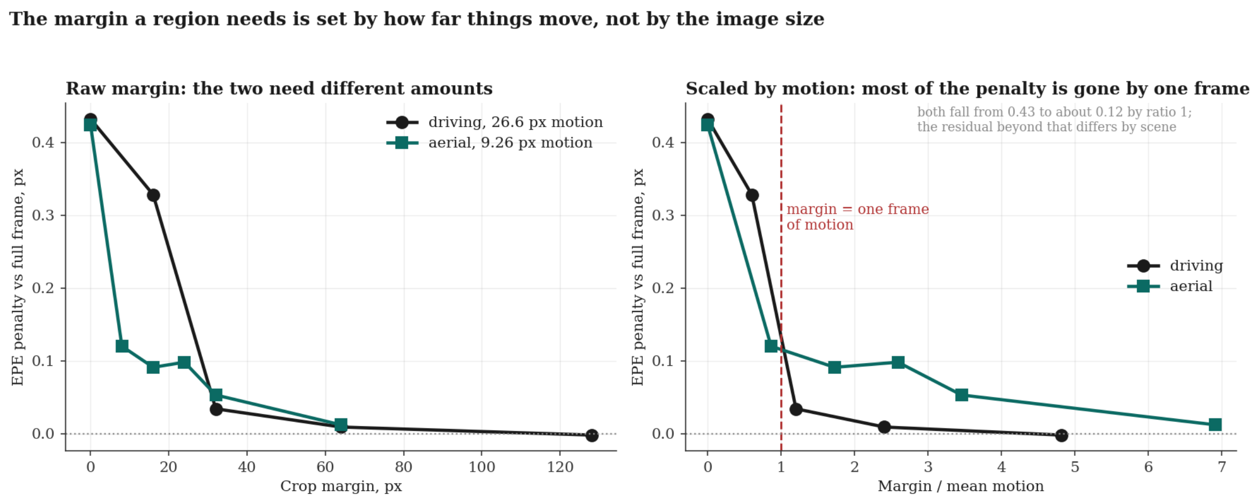

Fig. 3. Left: in raw pixels the two domains need different margins. Right: expressed as a multiple of mean inter-frame motion, both shed the bulk of the penalty by a ratio of one. Measured with the baseline network, since the effect belongs to the architecture family rather than to the decoder.

on its training domain for three properties the fixed-resolution design cannot express, from a model 13% smaller.

For an application that consumes one complete dense field per frame and nothing else, the baseline remains the correct choice: it is more accurate and 10% faster in that mode. The decoder earns its place when the consumer selects its own query points, revisits a frame, requires positions between pixels, or needs a confidence value with each correspondence.

We note that the selective-prediction result reframes the accuracy comparison. Mean error over all pixels is the metric on which the decoder loses, and it is the only operating point available to a model without confidence. Permitted to abstain, the same decoder is more than twice as accurate as the baseline over most of the frame.

VII. Limitations

Accuracy. The decoder is less accurate than the upsampler it replaces. The cause is structural: its finest input is at $1/8$ resolution, so the evidence available to it varies little within an 8×8 cell, whereas the baseline's upsampler reads the full-resolution frame directly. Three independent experiments support this diagnosis. Fourier positional encoding of the sub-cell offset produced no change, ruling out missing positional signal; querying below the input sampling rate matched bilinear upsampling exactly; and reducing the coarse resolution degraded both models identically. The indicated remedy, supplying the decoder with a full-resolution feature stem, has not been evaluated.

Sub-pixel querying. For the same reason, evaluation between input pixels is presently closer to interpolation than to inference. The interface is exact and the mechanism is real, but the additional information recoverable between pixels is small until the decoder receives full-resolution features.

Embedded validation. All latency figures are from a laptop GPU. No measurement on embedded hardware has been performed, so the edge-deployment premise remains a design target rather than a result.

Statistical resolution. One seed per configuration, with checkpoint-to-checkpoint variation of up to 0.038 px. Differences below approximately 0.05 px are not resolved by this study.

VIII. Conclusion

We replaced the fixed upsampler of a real-time optical flow network with an implicit decoder answering queries at arbitrary continuous coordinates, and measured what the substitution costs and provides. It costs 2.6% of mean accuracy on the training domain. It provides repeat queries at $27\times$ lower cost, calibrated per-query confidence that yields $2.2\times$ lower error over 80% of the frame under selective prediction, and evaluation between input pixels, from a model 13% smaller.

We additionally characterised region-directed inference for fast platforms, establishing that a cropped region requires a context margin proportional to inter-frame displacement, a relation that holds across a threefold difference in motion scale and can be evaluated from platform speed and frame rate in advance.

Future work is directed by the diagnosis above: a full-resolution feature stem addresses the identified accuracy deficit and would make sub-pixel querying substantive; embedded measurement is required before the deployment premise can be claimed; and evaluation against ground truth at twice the input resolution would test the arbitrary-resolution capability directly.

References

[1] Z. Zhang, A. Gupta, H. Jiang, and H. Singh, "NeuFlow v2: High-efficiency optical flow estimation on edge devices," *arXiv:2408.10161*, 2024.

[2] H. Jung, Z. Hui, L. Luo, H. Yang, F. Liu, S. Yoo, R. Ranjan, and D. Demandolx, "AnyFlow: Arbitrary scale optical flow with implicit neural representation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

Fig. 4. The same three scenarios on aerial sequences, whose mean inter-frame motion is a third of the driving case. The region geometry and the decoder configuration are identical to Fig. 1; only the domain changes, and no model in this study was trained on it.

[3] Z. Teed and J. Deng, "RAFT: Recurrent all-pairs field transforms for optical flow," in *Proc. European Conf. Computer Vision (ECCV)*, 2020.

[4] Yu *et al.*, "InfiniDepth: Implicit neural upsampling for monocular depth estimation," 2025.

[5] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] Y. Cabon, N. Murray, and M. Humenberger, "Virtual KITTI 2," *arXiv:2001.10773*, 2020.

[7] W. Wang, D. Zhu, X. Wang, Y. Hu, Y. Qiu, C. Wang, Y. Hu, A. Kapoor, and S. Scherer, "TartanAir: A dataset to push the limits of visual SLAM," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020.